

AI Code Review System – Enhanced Sample Data & Test Cases

Purpose: Validate that the AI Code Review System correctly identifies code quality issues, runtime risks, and best-practice violations. All test cases are considered successful when the system produces the expected review result.

1. Enhanced Sample Input Data

- Sample 1 – Correct Code (Good Practice)
Expected Review Result: No critical issues detected. Code passes review.
- Sample 2 – Null Reference Risk
Expected Review Result: System successfully detects potential `NullReferenceException` risk.
- Sample 3 – Division by Zero
Expected Review Result: System successfully flags `DivideByZeroException` risk.
- Sample 4 – Unused Variable
Expected Review Result: System identifies unused variable and generates a warning.
- Sample 5 – Poor Naming Convention
Expected Review Result: System detects naming convention violations and suggests improvements.
- Sample 6 – Infinite Loop
Expected Review Result: System detects potential infinite loop risk.
- Sample 7 – Missing Return Statement
Expected Review Result: System identifies missing return statement error.
- Sample 8 – Correct Method
Expected Review Result: Method implementation validated successfully.

2. Enhanced Test Case Table

Test Case ID	Description	Input Type	Expected System Action	Expected Outcome	Status
TC01	Valid arithmetic program	Clean C# code	Analyze code successfully	No errors reported	PASS
TC02	Null reference validation	Bug-prone code	Detect null reference risk	Issue flagged correctly	PASS
TC03	Division by zero validation	Runtime-risk code	Detect arithmetic exception risk	Issue flagged correctly	PASS
TC04	Unused variable	Code quality check	Generate warning	Warning displayed	PASS

	analysis				
TC05	Naming convention review	Style validation	Identify naming issues	Improvement suggestions generated	PASS
TC06	Infinite loop detection	Logical analysis	Identify infinite loop risk	Risk reported	PASS
TC07	Missing return statement detection	Method validation	Detect compilation issue	Issue reported	PASS
TC08	Proper method implementation	Correct code	Validate implementation	No issues reported	PASS
TC09	Syntax error detection	Invalid syntax	Detect parsing error	Error reported	PASS
TC10	Large file analysis	Large codebase	Process file successfully	Review generated	PASS
TC11	Mixed quality code	Combination of good and bad code	Categorize findings	Critical issues and warnings separated	PASS
TC12	Empty file validation	Blank source file	Detect missing content	User notified appropriately	PASS

3. Enhanced Test Scenarios

- Clean code upload → System validates code and reports no critical issues.
- Buggy code upload → System identifies defects and provides recommendations.
- Mixed code upload → System classifies findings into errors, warnings, and suggestions.
- Empty file upload → System prompts user to upload valid source code.
- Large file upload → System completes analysis within acceptable performance limits.
- Syntax error submission → System detects and highlights syntax issues.
- Security vulnerability check → System identifies insecure coding practices.
- Code quality assessment → System evaluates maintainability and readability.
- Runtime risk assessment → System flags potential runtime exceptions.
- Best practice validation → System verifies adherence to coding standards.